

IBM APPLIED DATA SCIENCE CAPSTONE

SpaceX Falcon 9 First-Stage Landing Prediction

A data science pipeline for predicting reusable booster landing outcomes

[Your Name]

Prepared for the IBM Applied Data Science Capstone • [Add date]

github.com/Nandhakishore-A/DATA_SCIENCE_CAPSTONE_PROJECT

Why this project, and what it found

● Business Problem

SpaceX advertises Falcon 9 launches at a fraction of competitors' prices, largely because it reuses the first-stage booster. Whether a competitor — or SpaceX itself — can predict if a booster will land successfully has direct cost implications.

● Objective

Build an end-to-end pipeline (API + web-scraped data collection → wrangling → SQL & visual EDA → geospatial analysis → dashboard → ML) to predict first-stage landing success from launch parameters.

● Key Results

90 historical Falcon 9 launches were analyzed. Overall observed landing success rate: 66.7%. Four classification models were tuned with GridSearchCV (10-fold CV).

● Best ML Model

Decision Tree Classifier: 87.5% cross-validated accuracy, 94.4% accuracy on the held-out test set — the highest test accuracy of the four models compared.

SpaceX, reusability, and why landing matters

1

SpaceX Background

- Falcon 9 is SpaceX's two-stage orbital rocket.
- This project's dataset (built in Data Collection API.ipynb) covers 90 past Falcon 9 launches retrieved from the SpaceX REST API.

2

Falcon 9 Reusability

- The Data Wrangling notebook labels each launch's Outcome (e.g. 'True ASDS', 'False Ocean', 'None None').
- A binary Class column (1 = booster landed, 0 = did not land) is derived from these outcome strings.

3

Why Landing Prediction Matters

- A landed booster can be refurbished and reused, lowering launch cost.
- Predicting landing outcome from launch parameters (payload, orbit, site) lets stakeholders estimate that cost advantage before a launch.

Two collection paths: REST API and web scraping

API Workflow — Data Collection API.ipynb

- GET request to `api.spacexdata.com/v4/launches/past`
- `pd.json_normalize()` converts the JSON response to a DataFrame
- Auxiliary functions `getBoosterVersion` / `getLaunchSite` / `getPayloadData` / `getCoreData` call the rocket, launchpad, payloads and cores sub-APIs to enrich each record
- Dataset filtered to Falcon 9 launches only; FlightNumber assigned 1–90

Web Scraping Workflow — Data Collection with Web Scraping.ipynb

- `requests.get()` on the static Wikipedia “List of Falcon 9 and Falcon Heavy launches” page
- `BeautifulSoup(response.content, 'html.parser')` parses the HTML
- `soup.find_all('table')` isolates the launch-record tables
- Column headers and row cells extracted into a pandas DataFrame

Dataset Summary (after wrangling)

90 launch records • 3 launch sites: CCAFS SLC 40 (55), KSC LC 39A (22), VAFB SLC 4E (13)

 No notebook cell-output screenshots were found in the ZIP for the API or web-scraping steps. Insert your own notebook screenshots here before submission.

Cleaning, labeling, and finalizing the dataset

Missing Values (raw API dataset)

- PayloadMass: 5 missing values
- LandingPad: 26 missing values (expected — not every launch attempted a ground/drone-ship landing)
- `df.isnull().sum()/df.count()*100` confirmed 0% missing in the final wrangled feature set

Feature Engineering

- `Outcome.value_counts()` produced 8 unique landing-outcome strings
- `bad_outcomes = {'False ASDS', 'False Ocean', 'False RTLS', 'None ASDS', 'None None'}`
- Binary Class label derived: 1 = landed, 0 = did not land

Final Dataset Description

90 rows total • LaunchSite: CCAFS SLC 40 (55), KSC LC 39A (22), VAFB SLC 4E (13)
Orbit: GTO (27), ISS (21), VLEO (14), PO (9), LEO (7), SSO (5), MEO (3), HEO (1), SO (1), GEO (1), ES-L1 (1)

Overall Class mean (success rate): 0.667 → 66.7% of launches in the dataset landed successfully

Two complementary approaches: SQL and Python

SQL EDA

Exploratory Data Analysis.ipynb

- Dataset loaded into an IBM Db2 (cloud) table Spacextbl
- Connected via the %sql ipython-sql magic
- 10 targeted business questions answered with direct SQL aggregation/filtering (totals, averages, rankings, subqueries)
- Why SQL: fast, declarative answers to specific numeric questions (e.g. total payload by customer) without loading the full frame into memory logic

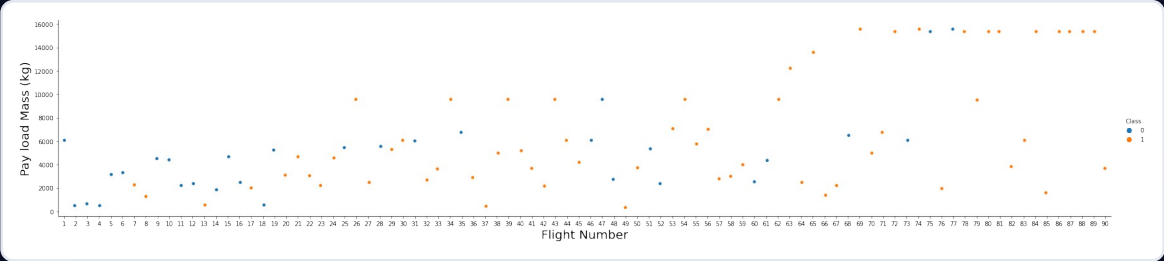
Python Visual EDA

EDA with Data Visualization.ipynb

- pandas + seaborn/matplotlib used to explore FlightNumber, PayloadMass, LaunchSite, Orbit vs. Class
- Categorical columns one-hot encoded and numeric columns cast to float64 for modeling
- Why Python/visual: scatter, bar and line plots reveal trends (e.g. success rate over time, clustering by orbit) that are hard to see in tabular SQL output alone

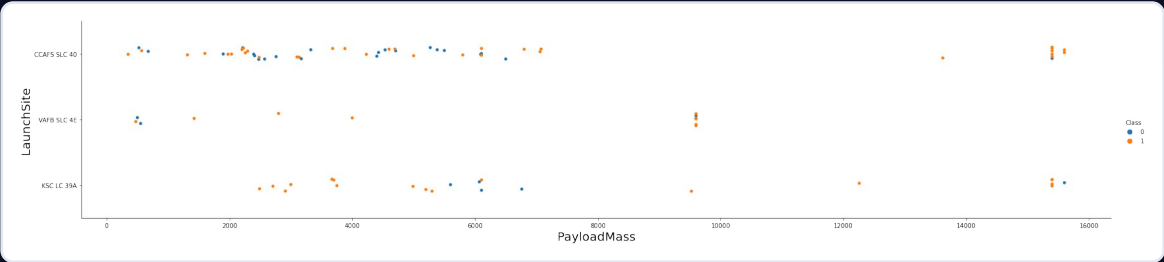
Scatter analysis — flight number, payload, site & orbit

Payload Mass vs Flight Number



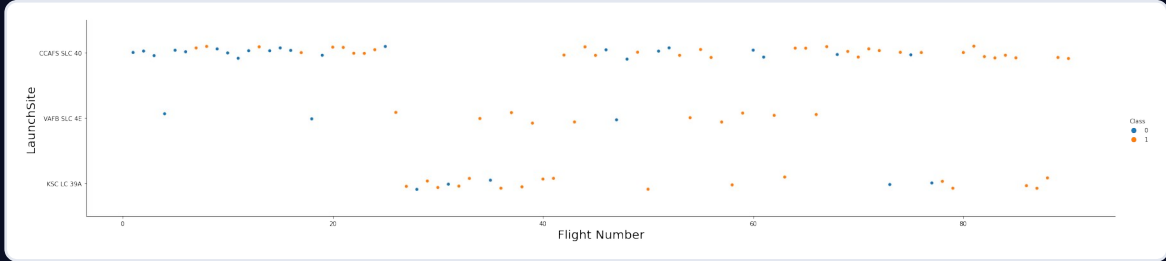
Later flights (higher FlightNumber) carry a wider range of payload mass, and successful landings (orange) become more frequent among later flights.

Payload Mass vs Launch Site



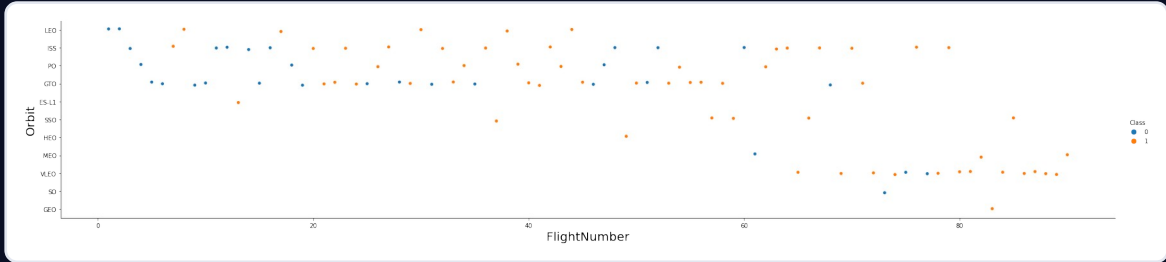
VAFB SLC 4E is used for a narrower payload-mass range than CCAFS SLC 40, which handles the heaviest payloads (~15,600 kg).

Flight Number vs Launch Site



KSC LC-39A only enters use from higher flight numbers onward; CCAFS SLC 40 is active across the full flight-number range.

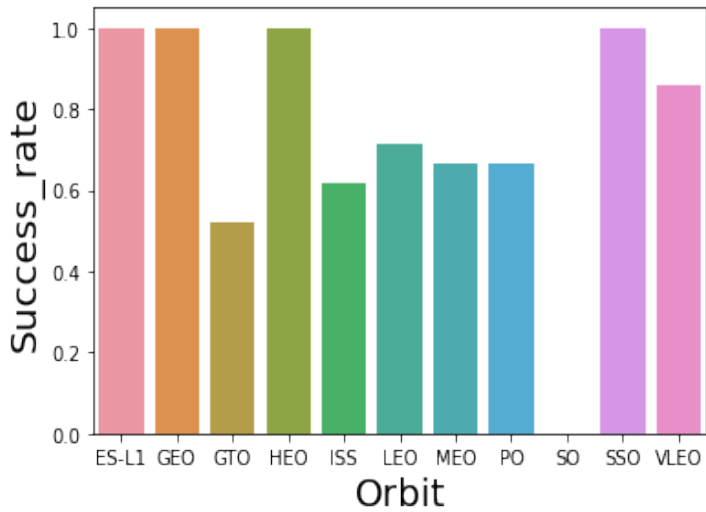
Flight Number vs Orbit Type



GTO and ISS orbits appear across the whole flight range; VLEO orbit launches are concentrated in later flights only.

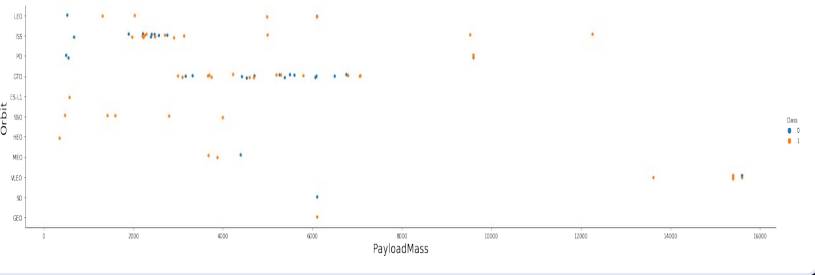
Success-rate distribution and yearly trend

Success Rate by Orbit (bar chart)



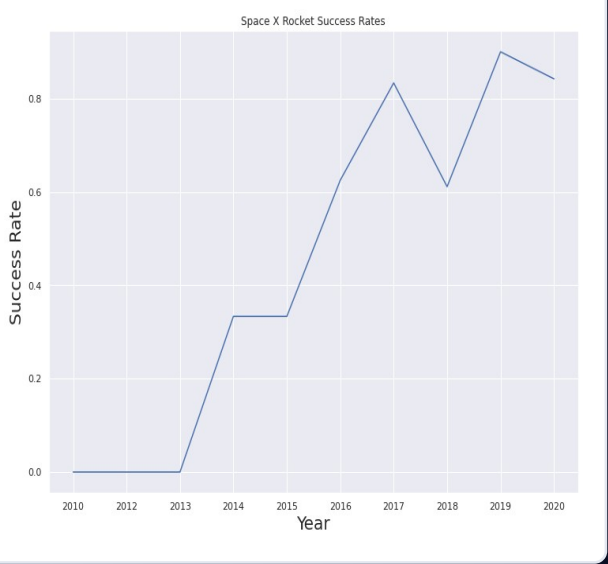
ES-L1, GEO, HEO and SSO orbits show a 100% observed success rate in this dataset; GTO is lowest at ~52%.

Payload Mass vs Orbit Type (scatter)



GTO and ISS orbits span a wide payload range; VLEO carries the heaviest payloads among frequently-used orbits.

Launch Success Rate by Year (line chart)



Success rate rose from 0% in 2010/2013 to roughly 83-90% by 2019-2020.

 *Pie Chart & Correlation Plot were NOT found as static outputs in any notebook. A pie chart exists only as an interactive Plotly component in the Dash app (Slide 11). Insert a correlation heatmap / pie chart here if you add one.*

10 business questions, answered directly against Spacextbl

#	Business Question	Result
1	Distinct launch sites	CCAFS LC-40, CCAFS SLC-40, KSC LC-39A, VAFB SLC-4E
2	Records where site LIKE 'CCA%' (first 5)	5 rows returned, e.g. F9 v1.0 B0003 — CCAFS LC-40 — 2010-06-04
3	Total payload mass, customer = NASA (CRS)	45,596 kg
4	Average payload mass, booster F9 v1.1	2,928 kg
5	First successful ground-pad landing date	2015-12-22
6	Boosters: drone-ship success, 4,000–6,000 kg payload	F9 FT B1022, B1026, B1021.2, B1031.2
7	Total successful vs. failed mission outcomes	100 success / 1 failure
8	Booster version(s) with maximum payload mass	15,600 kg — carried by 12 F9 B5 boosters
9	Failed drone-ship landings in 2015	F9 v1.1 B1012 (2015-01-10), B1015 (2015-04-14) — both CCAFS LC-40
10	Landing-outcome ranking, 2010-06-04 to 2017-03-20	No attempt (10) → ... → Precluded/drone ship (1)

Interpretation: NASA CRS missions carried a substantial payload share; F9 B5 boosters were used for the heaviest (15,600 kg) payloads; ground-pad landing success only began in Dec 2015; nearly all missions succeeded at their primary objective (100/101) even when the booster itself did not land.

Geospatial analysis of launch sites and proximities

Launch Site Map & Landing Sites

- `folium.Map()` centered on NASA Johnson Space Center [29.5597, -95.0831]
- `folium.Circle()` markers placed at each of the 3 launch sites
- `MarkerCluster()` groups per-launch markers, color-coded by success/failure at each site

Distance Analysis

- `calculate_distance(lat1, lon1, lat2, lon2)` implements the haversine formula
- Used to measure distance from each launch site to its nearest coastline, railway, highway, and the city of Cape Canaveral
- Markers + PolyLines annotate each measured distance on the map

Source notebook: `interactive_visual_analytics_with_folium_final.ipynb`

 No static screenshot of the Folium map was found in the ZIP — Folium maps render as interactive HTML/JS inside the notebook and were not exported as an image. Insert your own screenshot of the rendered map here.

Interactive launch-records dashboard — spacex_dash_app.py

Components

- Dropdown (site-dropdown): “All Sites” + each unique Launch Site
- Pie Chart (success-pie-chart): px.pie of successful launches by site, or success/fail split for one selected site
- RangeSlider (payload_slider): filters by payload mass, 0–10,000 kg
- Scatter Chart (success-payload-scatter-chart): px.scatter of Payload Mass vs. class, colored by Booster Version Category

Interactivity

- Callback 1: site-dropdown → success-pie-chart
- Callback 2: site-dropdown + payload_slider → success-payload-scatter-chart
- Selecting a site or dragging the payload slider re-renders both charts live

 No dashboard screenshot was found in the ZIP. The app also reads spacex_launch_dash.csv, which is not included — add that CSV and a live screenshot before submission.

Pipeline: features → split → scale → four tuned classifiers

Feature Selection

One-hot encoded categorical columns from EDA with Data Visualization.ipynb (Task 7), cast to float64 (Task 8).

Train / Test Split

train_test_split() used in SpaceX_Machine Learning Prediction_Part_5.ipynb to hold out a test set.

StandardScaler

Features standardized with StandardScaler().fit_transform() prior to model training.

Models compared (GridSearchCV, cv = 10, scoring = 'accuracy')

Logistic Regression

C: [0.01, 0.1, 1] • penalty: l2 • solver: lbfgs

SVM

C, gamma grid • kernel options incl. sigmoid

Decision Tree

criterion, max_depth, max_features, min_samples_leaf/split, splitter

KNN

algorithm • n_neighbors • p (distance metric)

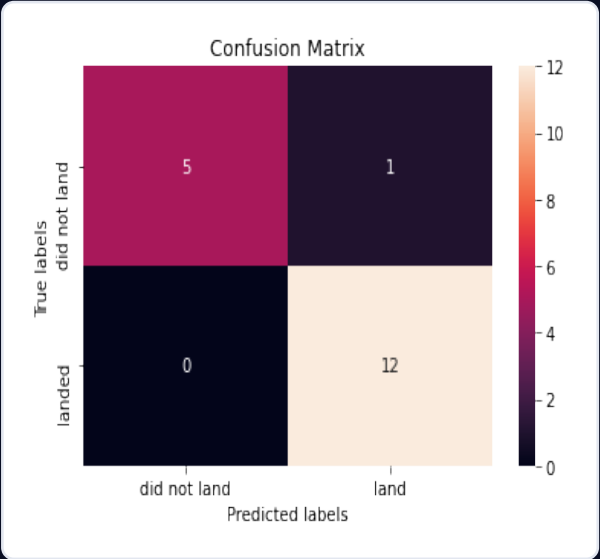
Decision Tree wins on the held-out test set

Accuracy Comparison

Model	CV Accuracy	Test Accuracy
Decision Tree	0.8750	0.9444
KNN	0.8482	0.8333
SVM	0.8482	0.8333
Logistic Regression	0.8464	0.8333

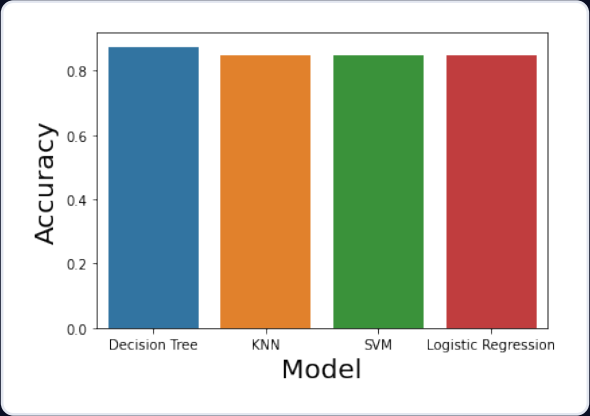
Best hyperparameters (Decision Tree): criterion = gini, max_depth = 4, max_features = sqrt, min_samples_leaf = 2, min_samples_split = 10, splitter = best

Confusion Matrix — Decision Tree (test set)



All 12 “landed” cases correctly classified; 5 of 6 “did not land” cases correctly classified (1 false positive).

Cross-Validated Accuracy by Model



⚠️ Classification Report (precision/recall/F1) and ROC Curve were NOT found in the notebook — only confusion matrices were generated. Add `sklearn.metrics.classification_report()` / `roc_curve()` output if required by your grader.

What the data shows

1

Historical first-stage landing success rate across all 90 launches is 66.7%.

2

Landing success rate climbed from 0% in 2010/2013 to roughly 83–90% by 2019–2020.

3

ES-L1, GEO, HEO and SSO orbits show a 100% success rate in this dataset, while GTO is lowest at ~52%.

4

Ground-pad landing success was not achieved until December 22, 2015 (SQL Task 5).

5

The Decision Tree model generalized best to the held-out test set (94.4%) despite ranking 2nd on cross-validation — its shallow depth (4) may reduce overfitting to the small (90-row) dataset.

6

CCAFS SLC 40 was used far more than the other two sites (55 of 90 launches, 61%).

CONCLUSION

From raw API data to a tuned landing-outcome classifier

Summary

This capstone implemented the full required pipeline — SpaceX REST API collection, Wikipedia web scraping, data wrangling, SQL-based EDA, Python visual EDA, Folium geospatial analysis, a Plotly Dash dashboard, and four tuned classification models — culminating in a Decision Tree classifier with 94.4% test accuracy.

Business Value

A reliable landing-outcome predictor lets a launch provider estimate whether a mission's first stage is likely to be reused — informing bid pricing and manifest planning ahead of launch.

Future Improvements

Add classification_report / ROC-AUC evaluation; capture Folium & Dash screenshots; supply the missing spacex_launch_dash.csv and raw dataset files; try ensemble models (Random Forest, XGBoost).

Bottom line: Decision Tree Classifier — 87.5% CV accuracy, 94.4% test accuracy — best model produced by this project.

Submission checklist, based on the ZIP contents found

✓ Found in ZIP

- ✓ GitHub Repository:
github.com/Nandhakishore-A/DATA_SCIENCE_CAPSTONE_PROJECT
- ✓ README.md (present, but minimal — replace with the generated README.md)
- ✓ Data Collection API.ipynb
- ✓ Data Collection with Web Scraping.ipynb
- ✓ Data Wrangling lab week 1.ipynb
- ✓ Exploratory Data Analysis.ipynb (SQL)
- ✓ EDA with Data Visualization.ipynb
- ✓ interactive_visual_analytics_with_folium_final.ipynb
- ✓ SpaceX_Machine Learning Prediction_Part_5.ipynb
- ✓ spacex_dash_app.py
- ✓ ds-capstone-template-coursera.pdf (assignment template)

✗ Missing — supply before submitting

- ✗ Dashboard & Folium map screenshots
- ✗ Raw / processed dataset CSVs (e.g. spacex_launch_dash.csv, required by spacex_dash_app.py)
- ✗ This presentation, exported as .pptx / .pdf (upload to the repo above)

Coverage against the required rubric items

Rubric Item	Status	Notes
Executive Summary	✓ Covered	Slide 2
Introduction	✓ Covered	Slide 3
Data Collection	✓ Covered	Slide 4 — API & scraping code confirmed; no notebook screenshots found
Data Wrangling	✓ Covered	Slide 5 — real missing-value and value_counts figures
EDA Methodology	✓ Covered	Slide 6
EDA with Visualization	⚠ Partial	Slides 7–8 — pie chart & correlation plot not present in any notebook
EDA with SQL	✓ Covered	Slide 9 — all 10 queries executed with real results
Folium Mapping	⚠ Partial	Slide 10 — code confirmed; no static screenshot in ZIP
Plotly Dash	⚠ Partial	Slide 11 — app code confirmed; no screenshot; CSV data file missing
ML Methodology	✓ Covered	Slide 12
ML Results	⚠ Partial	Slide 13 — confusion matrices present; classification report & ROC curve not found
Conclusion	✓ Covered	Slide 15